Actual Budget

Dossier de documentation fonctionnelle et d'architecture

Version analysée: export projet généré le 15/04/2026

Périmètre: application Actual, client web / desktop, moteur local, sync-server, domaines fonctionnels et parcours métier.

Objet: fournir une base de compréhension exhaustive avant évolution du produit.

Ce document reconstruit la logique fonctionnelle d'Actual à partir de l'export du dépôt, des pages de documentation embarquées et des handlers/API visibles dans le projet.

Sommaire

Table des matières - clic droit puis Mettre à jour le champ dans Word si nécessaire.

1. Résumé exécutif

Actual est un gestionnaire budgétaire local-first centré sur le budget, les registres de comptes et l'automatisation des écritures. La logique métier principale s'exécute côté client, sur une base SQLite locale, et non dans un backend distant central.

Le produit combine six briques fortes: budget mensuel, comptes et transactions, payees, règles, schedules et rapports. Le sync-server est optionnel et sert principalement à la synchronisation multi-appareils, à l'authentification et à certaines intégrations de bank sync.

Pour faire évoluer Actual, le bon angle d'entrée n'est pas uniquement l'interface: il faut comprendre l'enchaînement UI -> handlers/API -> moteur local -> base SQLite -> synchronisation éventuelle.

2. Méthode d'analyse et périmètre

La présente documentation est fonctionnelle et techno-fonctionnelle. Elle ne vise pas à documenter chaque composant React ni chaque table SQL individuellement, mais à fournir une vision exploitable du système pour cadrer une évolution produit.

Le contenu couvre: architecture d'exécution, classes fonctionnelles, objets métier, flux de transaction, création de comptes, import, règles, schedules, rapprochement, budgets, rapports, synchronisation et points d'extension.

Les diagrammes de séquence fournis couvrent les parcours métier principaux et les variantes structurantes de l'application. Ils constituent un socle quasi exhaustif du point de vue fonctionnel, sans prétendre modéliser chaque micro-interaction UI.

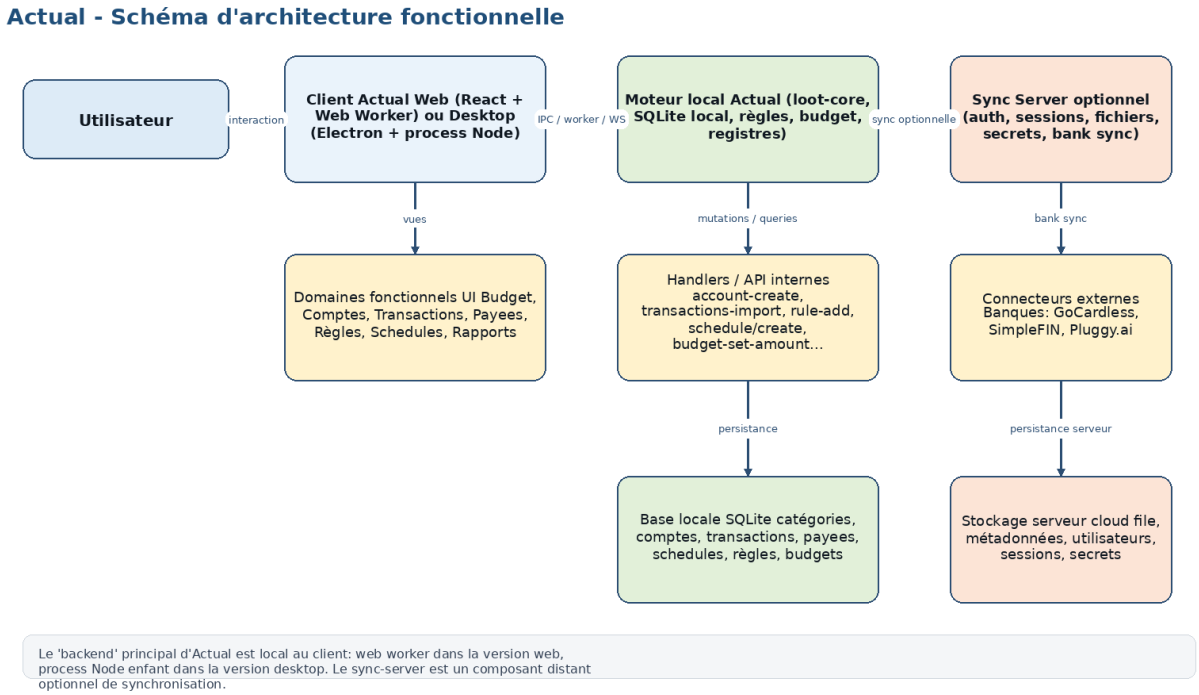
3. Vision produit et capacités couvertes

Positionnement - Actual supporte le budget par enveloppes, le budget de suivi traditionnel, l'usage offline, la synchronisation entre appareils via serveur, les transactions programmées, les règles transactionnelles, les rapports personnalisés et plusieurs fichiers budgétaires.

- Créer ou ouvrir un fichier budget local ou synchronisé.
- Gérer des comptes on-budget et off-budget.
- Importer des transactions de fichiers bancaires ou les saisir manuellement.
- Réconcilier un compte et verrouiller les opérations rapprochées.
- Normaliser les payees, apprendre des catégories et appliquer des règles automatiques.
- Programmer des écritures futures via schedules.
- Allouer des montants de budget par catégorie, avec carryover et arbitrages mensuels.
- Analyser la situation via rapports intégrés et rapports personnalisés.

4. Architecture d'ensemble

L'architecture fonctionnelle d'Actual repose sur une séparation entre une interface utilisateur, un moteur local de gestion budgétaire et, en option, un composant distant de synchronisation.



Lecture du schéma:

- Le client UI porte l'expérience utilisateur. En web, il s'agit d'une application React; en desktop, d'un client Electron.
- Le moteur local exécute la logique métier: base SQLite, registres, règles, budget, requêtes et handlers internes.
- Le sync-server n'est pas le coeur du calcul budgétaire; il gère surtout l'authentification, la persistance cloud des fichiers, certaines préférences et les secrets liés au bank sync.
- Les intégrations bancaires alimentent des transactions ou des soldes qui repassent ensuite par le flux standard d'import et de catégorisation.

Conséquence importante pour l'évolution:

Toute évolution fonctionnelle sérieuse doit être pensée à la fois dans l'UI, dans les handlers/API internes, dans les règles métier du moteur local et dans les impacts de persistance/synchronisation.

5. Cartographie des classes fonctionnelles

Classe fonctionnelle	Rôle	Objets principaux	Interactions clés
Pilotage budgétaire	Planifier les enveloppes mensuelles	BudgetMonth, Category, CategoryGroup	Transactions, carryover, reports

Comptes	Porter les registres et soldes	Account, balance, offbudget flag	Transactions, reconciliation, reports
Registre de transactions	Capturer les mouvements financiers	Transaction, split, transfer, cleared	Accounts, payees, categories, rules
Normalisation de la donnée	Nettoyer et enrichir les imports	Payee, imported payee, default category	Rules, learning, import
Automatisation	Appliquer des traitements déterministes	Rule, stage, condition, action	Import, manual edits, payees
Prévision	Projeter les écritures futures	Schedule, upcoming txn	Accounts, rules, budget planning
Synchronisation	Partager un budget entre terminaux	Sync file, session, cloud state	Client local, sync-server
Analyse	Exposer les indicateurs	Net worth, cash flow, custom report	Accounts, categories, transactions

Ces classes fonctionnelles sont la meilleure base pour découper des workstreams d'évolution: chacune possède ses propres règles, ses écrans, ses handlers et ses impacts de données.

6. Modèle métier de référence

Objets pivot

- Budget file - conteneur principal des données utilisateur.
- Account - compte financier, on-budget ou off-budget, ouvert ou fermé.
- Transaction - mouvement unitaire, manuel, importé, synchronisé, planifié ou généré.
- Payee - représentation normalisée de la contrepartie d’une transaction.
- Category / CategoryGroup - axes de ventilation budgétaire.
- Rule - ensemble ordonné de conditions et d’actions appliquées aux transactions.
- Schedule - modèle récurrent ou ponctuel permettant d’anticiper une écriture.
- BudgetMonth - état budgétaire d’un mois pour chaque catégorie.

Attributs structurants

- Account: nom, statut on/off budget, fermeture, balance.
- Transaction: date, montant, payee, imported payee, catégorie, état cleared/locked, note, type de liaison.

- Rule: stage (pre/default/post), ranking, conditions, actions.
- Schedule: nom, conditions, date/fréquence, montant approximatif ou fixe, auto-post ou approbation manuelle.

Contraintes de gestion

- Les comptes off-budget n'alimentent pas le budget et leurs transactions ne sont pas catégorisées.
- Une transaction splitée conserve un parent logique et plusieurs enfants ventilés.
- Un transfert est modélisé comme une transaction liée à un payee spécial de transfert.
- Les règles s'exécutent en ordre déterministe, avec priorité par stage puis par spécificité.

7. Gestion détaillée d'une transaction

La transaction est l'objet métier central. Le budget, les rapports, le rapprochement et l'automatisation se déduisent presque tous d'elle.

Cycle de vie standard

1. Création: manuelle, import de fichier, bank sync, schedule, transfert ou split.
2. Normalisation: résolution du payee, catégorie par défaut, enrichissement par règles.
3. Qualification: cleared, locked, liée à un schedule, issue d'un import, fusionnée, etc.
4. Ajustement: modification, recatégorisation, split, merge, transfert, suppression.
5. Consommation: budget mensuel, cash flow, net worth, filtres et rapports.

Comportements fonctionnels importants

- Déduplication à l'import: priorité à l'identifiant importé quand disponible; sinon heuristique date/montant/payee.
- Préférence de la transaction importée sur une saisie manuelle jumelle, notamment pour la justesse de la date.
- Possibilité de reimporter ou non les transactions précédemment supprimées selon le mode d'import.
- Possibilité de bulk edit, merge de doublons, split multi-catégories et transfert inter-comptes.

Lecture métier - Dans Actual, la transaction n'est pas seulement une ligne de compte. C'est aussi le point d'entrée de l'automatisation, de l'apprentissage des payees et de la consommation budgétaire.

8. Gestion détaillée des comptes

Le compte est le conteneur de registre. Il sert à regrouper les transactions, calculer un solde, piloter le rapprochement et alimenter certains rapports.

Typologie

- On-budget: participe à la trésorerie disponible et à l'enveloppe budgétaire.
- Off-budget: suit un actif, une dette ou un investissement sans affecter les enveloppes.

- Ouvert / fermé: un compte fermé reste consultable dans la hiérarchie.

Fonctions de gestion

- Création locale ou via liaison à un provider de bank sync.
- Nom, note, statut on/off budget, balance initiale.
- Import, export, filtre, recherche, affichage du running balance.
- Fermeture avec transfert de solde, réouverture, suppression forcée.

Rapprochement

Le rapprochement compare le ledger Actual au relevé bancaire. Les écritures passent par les états pending -> cleared -> locked. Une fois le différentiel à zéro, les écritures rapprochées peuvent être verrouillées.

9. Budgeting et enveloppes

Le budget est organisé par mois, groupes de catégories et catégories. Les trois valeurs visibles par cellule sont Budgeted, Spent et Balance.

- Budgeted: montant alloué au mois pour la catégorie.
- Spent: consommation observée sur le mois.
- Balance: reliquat budgétaire, tenant compte du reste du mois précédent selon les règles du produit.
- Accélérateurs: copie du mois précédent, moyennes 3/6/12 mois, remise à zéro, notes de cellule.
- Paramètres complémentaires: carryover, hold for next month, templates/goals selon fonctionnalités avancées.

Angle de conception - Le budget Actual n'est pas une simple grille de saisie. C'est une projection calculée en continu depuis les catégories, les transactions réelles et les options de report de solde.

10. Payees, catégorisation et règles

Les payees servent d'ancre de normalisation. Le système cherche d'abord à retrouver un payee exact; sinon il essaie d'en déduire un via les règles; à défaut il crée un nouveau payee.

Règles

- Les règles traitent les transactions importées ou synchronisées.
- Une règle possède des conditions et des actions; plusieurs règles peuvent s’appliquer à la même transaction.
- Le moteur exécute les stages pre, default puis post.
- Les règles sont rangées de la moins spécifique à la plus spécifique à l’intérieur d’un stage.
- Les actions peuvent modifier catégorie, payee, notes, cleared, compte, date, montant et textes de notes.

Apprentissage automatique assisté

- Renommer un payee peut déclencher une proposition de règle de renommage future.
- Catégoriser plusieurs fois un même payee peut créer ou mettre à jour une règle de catégorie.
- L’apprentissage peut être désactivé pour un ou plusieurs payees.

Gouvernance des payees

- Renommer, supprimer, fusionner, marquer en favori, associer une catégorie par défaut.
- Les transfer payees sont des payees spéciaux permettant de créer des transferts automatiques.

11. Schedules et anticipation budgétaire

Le schedule est à la fois un outil d'automatisation et un outil de prévision. Sa valeur principale est de rendre visibles les dépenses futures et d'aider l'utilisateur à préparer son budget à l'avance.

- Occurrence unique ou récurrente.
- Date fixe, jours multiples, périodicités variées, déplacement avant/après week-end.
- Montant fixe ou approximatif.
- Auto-post dans le registre ou simple workflow d’approbation.
- Affichage upcoming configurable selon une fenêtre temporelle choisie.
- Possibilité de relier règles et schedules pour catégoriser ou annoter automatiquement.

12. Rapports et analyse

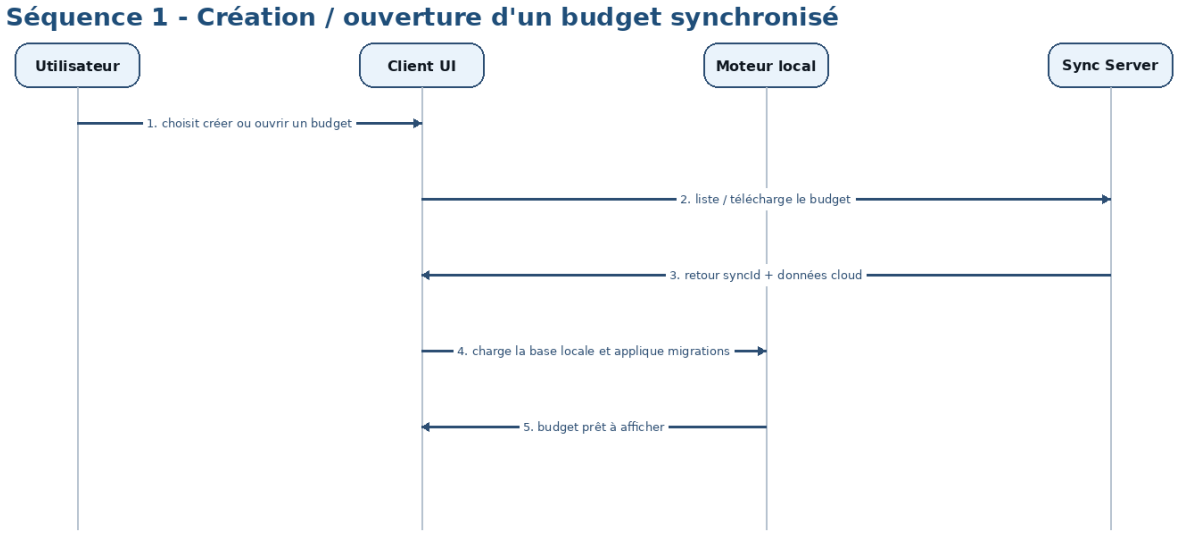
Actual expose des rapports standard et des rapports personnalisés. Les rapports standard donnent une lecture transverse du patrimoine et des flux; les rapports personnalisés permettent des analyses fines par catégorie, période ou filtre.

- Net worth: soldes agrégés actifs moins dettes.
- Cash flow: lecture des revenus et dépenses sur les comptes budgétaires.
- Custom reports: vues analytiques tabulaires ou graphiques sur les dépenses et revenus.
- Les rapports consomment essentiellement comptes, transactions, catégories et périodes.

13. Séquences fonctionnelles majeures

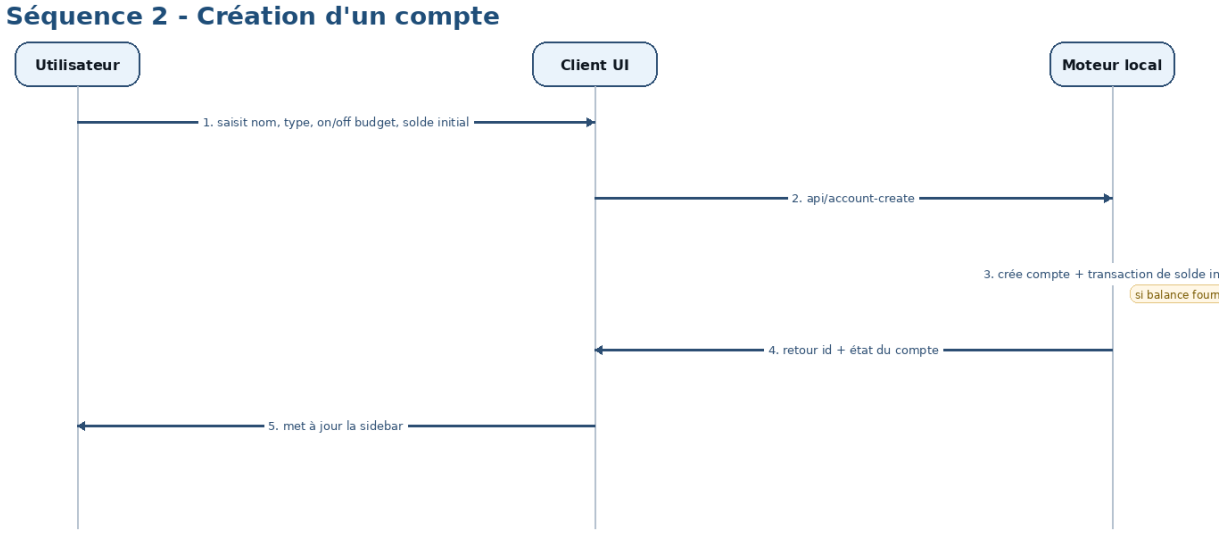
Les diagrammes ci-dessous couvrent les parcours structurants du produit. Ils sont volontairement orientés décision fonctionnelle plutôt qu'implémentation bas niveau.

Séquence 1 - Création / ouverture d'un budget synchronisé



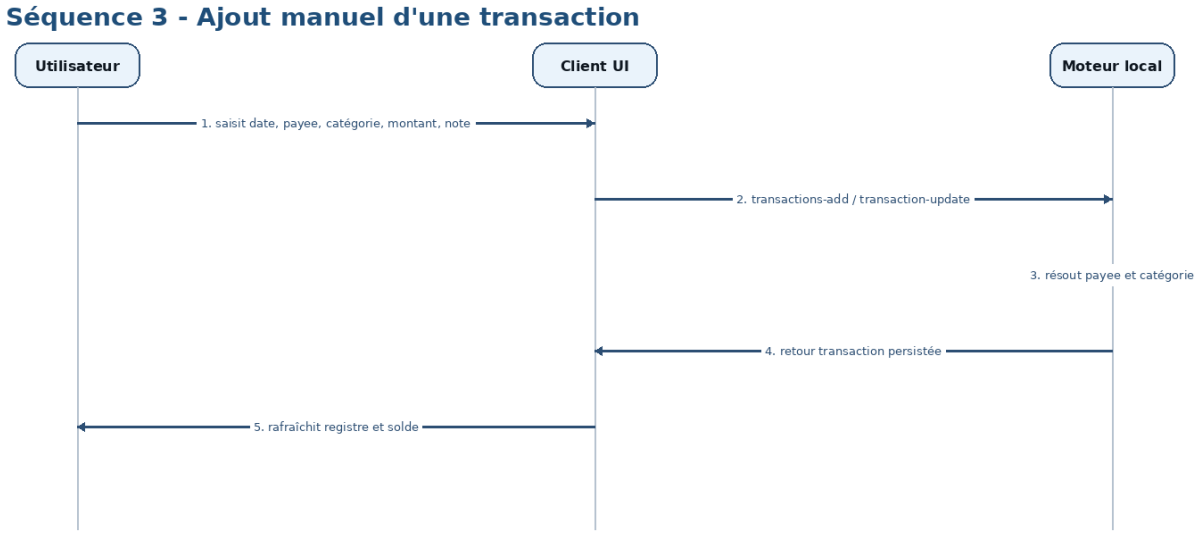
- Ce flux montre le chemin nominal et les responsabilités respectives des composants.
- Il met en évidence les objets métier concernés et le point exact où une future évolution devrait se brancher.

Séquence 2 - Création d'un compte



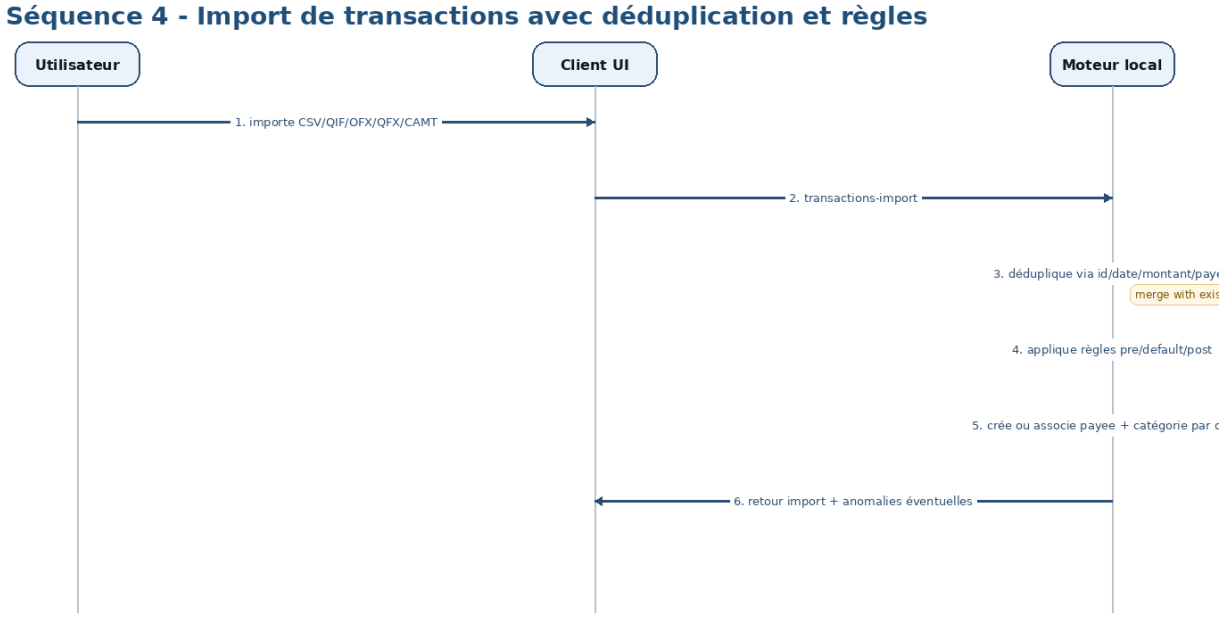
- Ce flux montre le chemin nominal et les responsabilités respectives des composants.
- Il met en évidence les objets métier concernés et le point exact où une future évolution devrait se brancher.

Séquence 3 - Ajout manuel d'une transaction



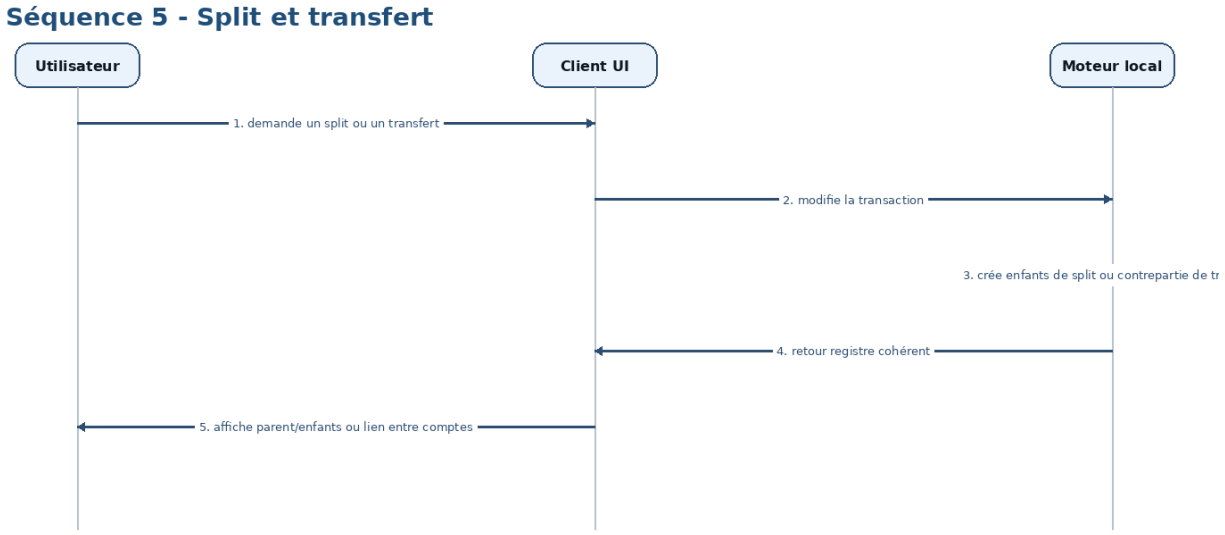
- Ce flux montre le chemin nominal et les responsabilités respectives des composants.
- Il met en évidence les objets métier concernés et le point exact où une future évolution devrait se brancher.

Séquence 4 - Import de transactions avec déduplication et règles



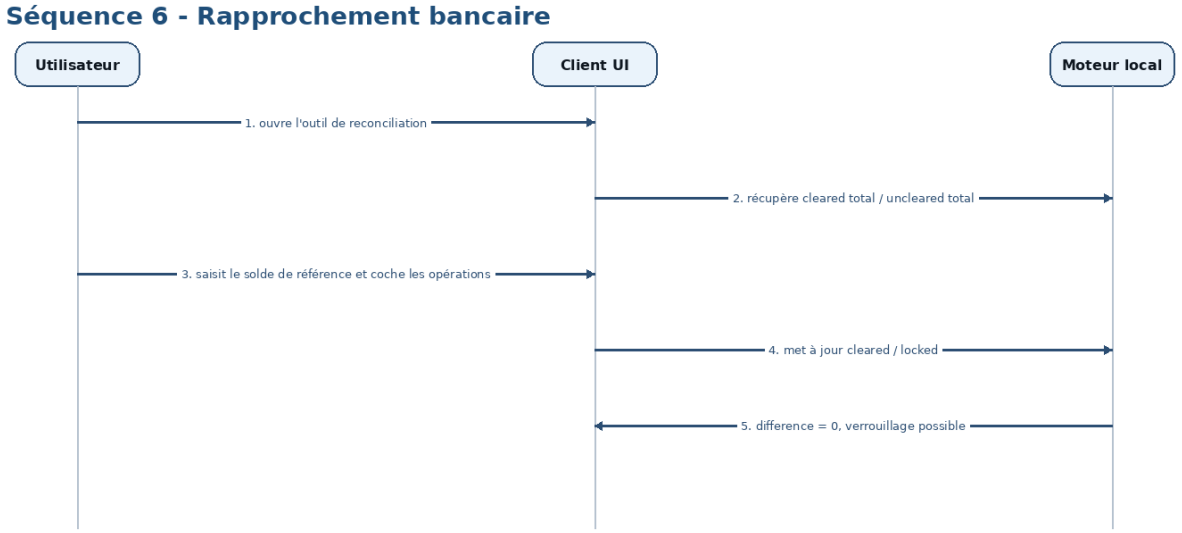
- Ce flux montre le chemin nominal et les responsabilités respectives des composants.
- Il met en évidence les objets métier concernés et le point exact où une future évolution devrait se brancher.

Séquence 5 - Split et transfert



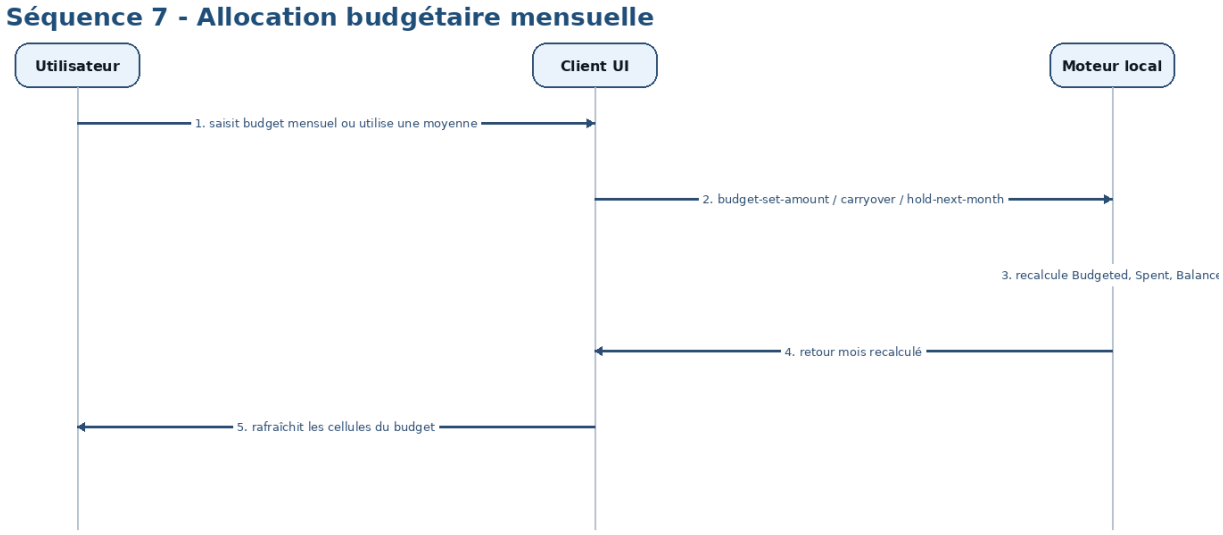
- Ce flux montre le chemin nominal et les responsabilités respectives des composants.
- Il met en évidence les objets métier concernés et le point exact où une future évolution devrait se brancher.

Séquence 6 - Rapprochement bancaire



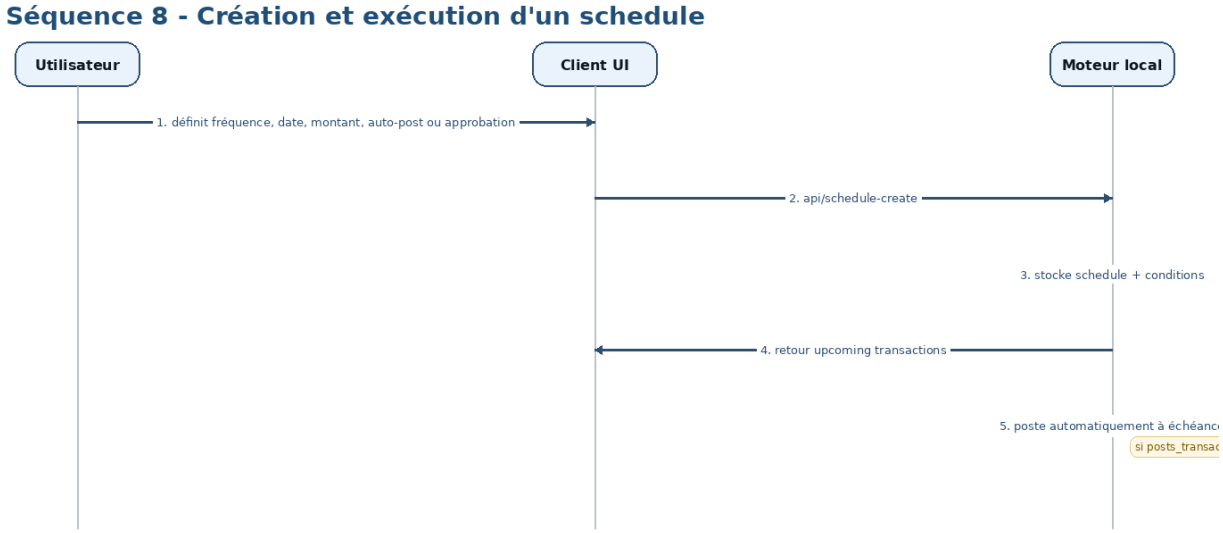
- Ce flux montre le chemin nominal et les responsabilités respectives des composants.
- Il met en évidence les objets métier concernés et le point exact où une future évolution devrait se brancher.

Séquence 7 - Allocation budgétaire mensuelle



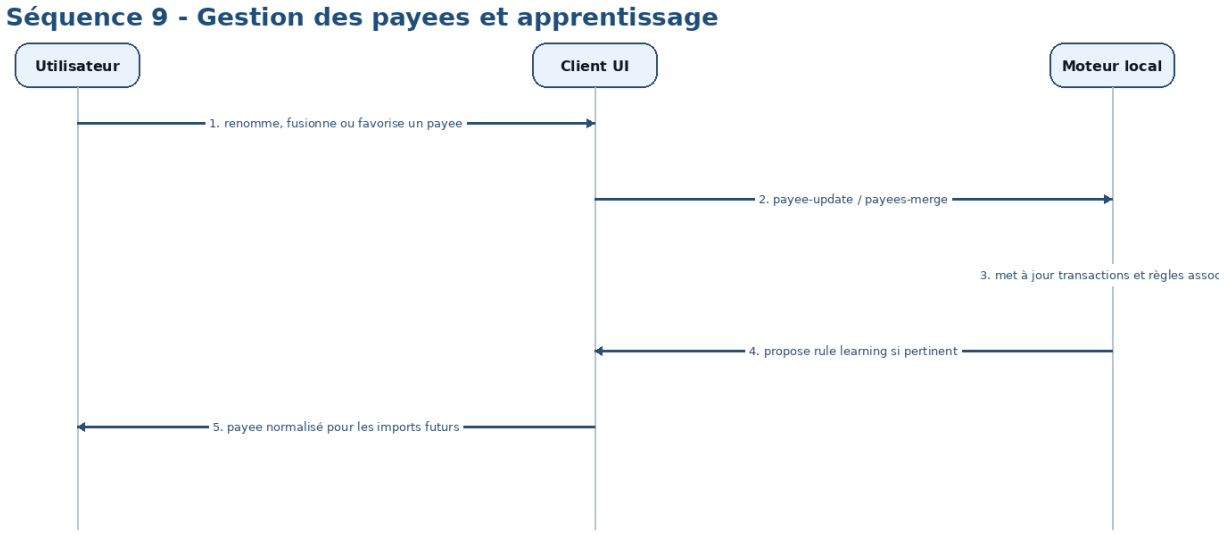
- Ce flux montre le chemin nominal et les responsabilités respectives des composants.
- Il met en évidence les objets métier concernés et le point exact où une future évolution devrait se brancher.

Séquence 8 - Création et exécution d'un schedule



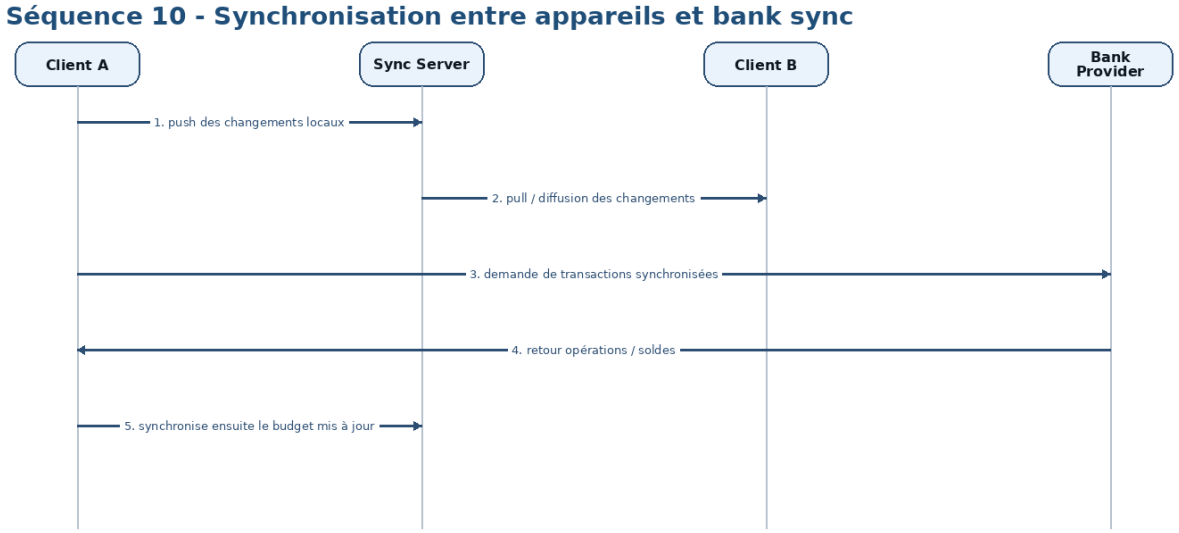
- Ce flux montre le chemin nominal et les responsabilités respectives des composants.
- Il met en évidence les objets métier concernés et le point exact où une future évolution devrait se brancher.

Séquence 9 - Gestion des payees et apprentissage



- Ce flux montre le chemin nominal et les responsabilités respectives des composants.
- Il met en évidence les objets métier concernés et le point exact où une future évolution devrait se brancher.

Séquence 10 - Synchronisation entre appareils et bank sync



- Ce flux montre le chemin nominal et les responsabilités respectives des composants.
- Il met en évidence les objets métier concernés et le point exact où une future évolution devrait se brancher.

14. Points d'attention pour une évolution future

- Ne pas traiter le sync-server comme source unique de vérité fonctionnelle: la vérité métier est d’abord locale.

- Toute évolution des transactions doit être vérifiée sur les impacts payees/règles/budget/reports/reconciliation.
- Les comptes off-budget doivent rester isolés des mécanismes de catégorisation et d'enveloppes.
- Les règles sont un multiplicateur fonctionnel: toute nouvelle donnée importée devrait idéalement pouvoir être ciblée par conditions/actions.
- Les schedules sont aussi un outil d'anticipation; les faire évoluer uniquement comme automatisation serait réducteur.
- Le mode multi-appareils impose de penser conflits, synchronisation, versioning et migrations.

Recommandation - Pour faire évoluer Actual vers de nouveaux usages, je recommande de conserver un noyau transactionnel déterministe, puis d'ajouter des capacités par couches: enrichissement d'import, aide à la catégorisation, moteur budgétaire, enfin projections et recommandations.

15. Proposition de structure documentaire cible pour poursuivre le projet

Je recommande de conserver ce livrable comme volume 0, puis de le décliner en une base documentaire d'évolution du produit.

Document cible	Contenu	Usage
01_vision-produit.md	positionnement, objectifs, principes	pilotage
02_architecture-fonctionnelle.md	domaines, flux, diagrammes	cadrage
03_modele-metier.md	objets, invariants, règles	conception
04_parcours-utilisateur.md	séquences par cas d'usage	UX et QA
05_backlog-evolutions.md	epics, stories, impacts	delivery
06_matrice-impacts.md	features x modules x données	analyse de risque

Cette structure est adaptée si vous souhaitez transformer la compréhension actuelle en chantier d'évolution maîtrisé, avec Codex ou un autre agent de développement.

Annexe - Références internes utilisées

- Pages de documentation embarquées: accounts, budgeting/rules, schedules, tour, reports, transactions/importing, transactions/payees, getting-started/sync.
- Handlers/API visibles dans l'export: account-create, account-update, categories-get, payee-create/update/delete, rules-get, rule-create/update/delete, schedules-get/create/update, transactions-import/update, budgets set-amount/carryover/hold.
- Éléments sync-server: migrations, account-db, auth, secrets, health-check, multi-user, intégrations GoCardless/SimpleFIN/Pluggy.ai.